江西農業大學

JIANGXI AGRICULTURAL UNIVERSITY

# 本科毕业论文

题目： 基于微服务架构的高校智能选课系统设计与实现


学 院： 软件学院

姓 名： 彭益举

学 号： 6020222035

专 业： 软件工程

班 级： 2206

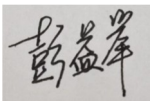
指导教师： 李佳航


答辩日期： 2026 年 04 月 19 日

# 江西农业大学

# 本科毕业论文原创性声明

本人郑重声明：所呈交的本科毕业论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本设计不包含任何其他个人或集体已经发表或撰写过的研究成果。对设计所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名： 

日　期：2026/3/11

# 目 录

第 1 章 绪论.....1
1.1 课题研究背景及意义.....1
1.2 国内外研究现状.....1
1.3 论文的主要研究内容.....2
1.4 论文的组织结构.....3
1.5 本章小结.....3
第 2 章 相关技术介绍.....5
2.1 相关技术.....5
2.1.1 Spring Boot 与 Spring Cloud Alibaba 微服务架构.....5
2.1.2 Nacos 服务注册与配置中心.....5
2.1.3 Redis 缓存与 Redisson 分布式锁.....5
2.1.4 RabbitMQ 消息队列与异步处理.....5
2.1.5 Sa-Token 统一鉴权框架.....6
2.1.6 MyBatis-Plus 数据访问框架.....6
2.1.7 React 与 Ant Design Pro 前端框架.....6
2.2 开发环境.....6
2.2.1 开发工具.....6
2.2.2 运行环境.....7
2.3 本章小结.....7
第 3 章 系统分析.....8
3.1 可行性分析.....8
3.1.1 技术可行性.....8
3.1.2 用户操作体验可行性.....8
3.1.3 开发成本可行性.....8
3.2 功能需求分析.....9
3.2.1 功能描述.....9
3.2.2 管理员用例图.....9
3.2.3 用户综合用例图.....10
3.3 本章小结.....11
第 4 章 系统设计.....12
4.1 系统架构设计.....12

4.2 总体结构设计.....12
4.2.1 用户服务层.....13
4.2.2 核心业务层.....13
4.2.3 数据支撑层.....13
4.3 功能模块设计.....14
4.3.1 智能课程推荐模块.....14
4.3.2 选课模块.....14
4.3.3 课程冲突检测模块.....15
4.3.4 系统管理模块.....15
4.3.5 消息通知与网关安全模块.....15
4.4 数据库设计.....15
4.4.1 概念设计.....15
4.4.2 逻辑设计.....16
4.5 本章小结.....18
第 5 章 系统实现.....20
5.1 智能课程推荐模块.....20
5.1.1 推荐算法设计与实现.....20
5.1.2 推荐课程页面.....20
5.2 选课模块的实现.....21
5.2.1 高并发选课核心实现.....21
5.2.2 选课操作页面.....21
5.2.3 选课结果通知页面.....22
5.2.4 统一登录与网关转发流程.....23
5.3 课程冲突检测模块.....24
5.3.1 冲突检测算法实现.....24
5.4 系统管理模块的实现.....25
5.4.1 学生管理页面.....25
5.4.2 课程管理页面.....25
5.4.3 教师管理页面.....26
5.5 消息通知模块的实现.....27
5.6 系统安全模块的实现.....28
5.6.1 Sa-Token 登录与 Token 管理.....28
5.6.2 网关层鉴权过滤器.....28

5.7 本章小结.....29
第 6 章 系统测试.....30
6.1 系统测试设计.....30
6.1.1 功能测试设计.....30
6.1.2 性能测试设计.....30
6.1.3 界面测试设计.....31
6.2 系统测试执行.....31
6.2.1 功能测试执行.....31
6.2.2 性能测试执行.....32
6.2.3 界面测试执行.....33
6.2.4 执行结果汇总.....34
6.3 Bug 修复与回归测试.....34
6.4 系统测试总结.....34
第 7 章 总结与展望.....36
7.1 总结.....36
7.2 展望.....36

# 摘 要

随着高等教育规模持续扩大，高校选课系统在学期初选课高峰期面临严峻的高并发压力。传统单体架构难以应对大规模并发访问、课程资源动态分配以及个性化推荐等多元需求，系统稳定性、数据一致性和用户体验均存在明显不足。大量高校在选课开放后的前几分钟内即出现服务器响应超时甚至宕机的情况，严重影响学生的正常选课流程，暴露出传统系统架构在突发高并发场景下的脆弱性。

本文设计并实现了一套基于微服务架构的高校智能选课系统，技术选型上以 Spring Boot 2.6.13 和 Spring Cloud Alibaba 为核心，Nacos 统一管理服务注册与配置，Spring Cloud Gateway 对外提供路由入口，Sa-Token 处理用户鉴权。选课并发控制方面，引入 Redisson 分布式锁保护库存扣减临界区，通过 RabbitMQ 将选课请求异步化从而平滑流量峰值，同时在选课服务内部记录 requestId 与处理状态，便于结果回查与异常排查。课程推荐模块根据学生专业字段为课程打分，专业对口课程给 90 分，通用选修课程给 50 分，降序取前 N 条作为推荐列表，并附上推荐理由文字；时间冲突检测则在原课程时间窗口基础上两端各延展 15 分钟，模拟课间转场所需时间，再与已选课程逐一比对。前端选用 React 结合 Ant Design Pro 构建管理界面，数据层使用 MyBatis-Plus 简化了大量重复的 CRUD 操作。

本文通过接口测试、界面测试和并发压力测试对系统进行了验证。测试结果表明，该系统能够较好地完成登录鉴权、课程推荐、在线选课、消息通知等核心业务流程，并在选课高峰场景下保持较稳定的运行状态，说明所采用的微服务架构、异步消息处理和并发控制方案具有一定的工程应用价值。

**关键词：**微服务；智能推荐；高并发；选课系统；冲突检测

# Abstract

With the continuous expansion of higher education, the university course selection system faces severe high-concurrency pressure at the beginning of each semester. Traditional monolithic architectures are inadequate to handle large-scale concurrent access, dynamic course resource allocation, and personalized recommendation requirements, leading to obvious deficiencies in system stability, data consistency, and user experience.

This paper designs and implements an intelligent course selection system based on microservice architecture, solving three core problems: system stability and data consistency under high concurrency, personalized intelligent course recommendation, and automatic time-conflict detection and warning. The system uses Spring Boot 2.6.13 with Spring Cloud Alibaba, Nacos for service registration and configuration, Spring Cloud Gateway for unified routing and authentication, and Sa-Token for unified authentication management. Redisson distributed locks protect stock deduction, RabbitMQ handles asynchronous request processing and notifications, and enrollment logs record request status for tracing. The recommendation module scores courses based on major match, assigning 90 points to matched courses and 50 points to general electives. The conflict detection module uses a time-window algorithm with 15-minute buffer thresholds. The frontend is built with React and Ant Design Pro, and the backend data layer uses MyBatis-Plus.

The testing results indicate that the system can correctly complete core functions such as authentication, recommendation, course selection, and message notification, while maintaining stable operation under peak course-selection workloads. This demonstrates that the adopted microservice architecture, asynchronous message processing, and concurrency control strategy have practical engineering value.

**Key Words:** Microservices; Intelligent Recommendation; High Concurrency; Course Selection; Conflict Detection

# 第 1 章 绪论

## 1.1 课题研究背景及意义

在高等教育数字化转型的背景下，选课系统作为支撑教学管理的关键基础设施，其性能优化和稳定性保障面临着全新技术挑战。特别是在春季或秋季学期选课高峰期，系统需要同时应对大规模并发访问、课程资源动态分配以及个性化推荐等多重需求。已有微服务架构与分布式系统相关研究表明，当业务访问量在短时间内集中爆发时，系统架构的伸缩能力、服务治理能力以及并发控制机制会直接影响整体稳定性与用户体验[1][2]。

传统选课系统通常采用集中式单体架构，将所有功能模块耦合在同一个部署单元中。这种架构在并发请求剧增时极易产生单点瓶颈，任何一个模块的性能下降都会影响整个系统的响应速度。更重要的是，传统系统普遍缺乏智能化的个性化推荐功能，学生需要在数百门可选课程中手动逐一筛选，效率低下，且容易遗漏与自身专业高度相关的优质课程。与此同时，课程时间冲突预警机制的缺失也导致学生在选课完成后才发现已选课程时间相互重叠，不得不重新进行选课调整，造成不必要的困扰。

为解决上述问题，本文引入 Spring Cloud Alibaba 微服务架构，通过服务解耦和独立部署实现功能模块的高内聚低耦合，显著提升系统的可扩展性与可维护性。同时，基于学生专业信息的智能课程推荐机制和时间窗口冲突检测算法为学生提供更加便捷、智能的选课体验，充分顺应了国家关于推进智慧校园建设的相关政策导向，为高校教务管理数字化转型提供有力技术支撑。

本研究的价值体现在以下三个层面：第一，技术创新层面，Redisson 分布式锁、RabbitMQ 消息队列、Redis 缓存的综合应用构建了高并发场景下的多重防护体系，显著提升了系统的抗压能力；第二，功能完善层面，智能推荐与冲突检测功能填补了传统系统的功能空白，显著提升了学生的选课体验与选课效率；第三，工程实践层面，本系统为高校教务信息化改造提供了可落地的微服务架构参考方案，具有较强的推广应用价值。

## 1.2 国内外研究现状

国内外关于高校选课系统的研究已取得一定进展，但在微服务架构、智能推荐与高并发处理的综合应用方面仍存在较大提升空间。

在国内研究方面，现有文献已围绕微服务拆分、缓存与分布式锁、消息中间件幂等控制等主题展开了较多讨论。曾佳从 Spring Cloud 微服务架构角度分析了业务系统拆分后的服务协同问题[1]；李俊俊等结合 Kubernetes 场景研究了服务资源调度与集群弹性能力[2]；王栋柱等讨论了基于 Redis 的高性能分布式锁设计方法

[6]；张旭东等则从中间件视角研究了分布式系统幂等性控制[8]。这些成果为高校选课系统在高并发场景下的稳定运行提供了可借鉴的工程思路。

在国外研究方面，近年来微服务架构、Kubernetes 部署治理和系统安全分析工具持续成熟。Abd-Elwahab 等从企业架构视角总结了微服务在基础设施优化中的可扩展性优势[3]；Augustyn 等围绕 Kubernetes 场景下的水平伸缩参数调优展开研究，为高并发业务系统的弹性部署提供了可参考的性能治理思路[4]。这些研究说明，采用微服务与云原生技术构建业务系统已成为当前的重要方向。

通过梳理国内外研究现状可以看出，现有高校选课系统研究仍存在若干共性不足。高并发防超卖方面，不少方案仍依赖数据库层面的乐观锁或悲观锁，在超高并发峰值下容易出现明显的性能瓶颈；课程推荐方面，部分研究倾向于采用深度学习等复杂模型，但这类方案对数据规模和服务器资源要求较高，在多数高校现有条件下推广成本较大；冲突检测功能方面，已有研究多停留在分钟级时间点比较，较少考虑学生在不同教学楼之间转场所需的现实时间。基于上述分析，本文选择以 Redisson 分布式锁配合 RabbitMQ 削峰的方式应对并发问题，以轻量、可解释的规则实现课程推荐，并在冲突检测中引入 15 分钟缓冲阈值，以兼顾技术可行性与工程落地成本。

## 1.3 论文的主要研究内容

本文的研究内容可以从以下几个模块来描述，各模块相互配合，共同构成一个完整的选课服务闭环。

（1）智能课程推荐模块。该模块以学生专业字段为核心匹配维度，对候选课程进行规则化评分。本专业相关课程赋予 90 分，通用选修课程赋予 50 分，再按分值降序返回前若干条推荐结果。每条推荐数据均附带解释性文字，例如"该课程与你的专业相关"，以增强推荐结果的可理解性。具体实现中，推荐服务通过 OpenFeign 跨服务调用选课微服务的分页接口获取课程数据，默认返回 5 条，前端也可通过 limit 参数自定义数量。

（2）高并发选课防护模块。针对选课高峰期的高并发问题，系统将选课服务从主业务服务独立部署，通过 Redisson 分布式锁确保同一课程在同一时刻只有一个请求进入库存扣减临界区。引入 RabbitMQ 消息队列将选课请求异步化处理，在峰值期间实现流量削峰填谷；同时在选课服务内部记录 requestId 和处理状态，用于结果回查与异常排查。

（3）课程时间冲突检测模块。系统实现了基于时间窗口扩展的冲突检测算法，以课程默认时长 2 小时为基准，在课程开始与结束时间各设置 15 分钟的转场缓冲阈值，形成扩展时间区间。通过获取学生已选课程列表，逐一与待选课程扩展区间进行交叉比对，并根据时间差生成差异化冲突原因描述，为学生提供清晰的冲

突预警信息。

（4）系统管理模块。涵盖学生管理、教师管理、课程管理三个子模块，按管理员、教师、学生三类用户区分功能入口与操作边界，支持课程二级审批流程（教师创建→管理员审核→上线可选），确保课程信息的规范性与教学质量标准。

（5）消息通知模块。选课结果无论成功还是失败均通过 RabbitMQ 消息队列异步推送至站内消息中心，通知内容以结果提示和失败原因说明为主，确保学生及时获取选课结果与课程变动信息。

（6）网关与安全模块。基于 Spring Cloud Gateway 实现统一路由转发与请求过滤，Sa-Token 框架负责登录态校验与会话管理，Nginx 通过 SSL/TLS 加密保障数据传输安全。

## 1.4 论文的组织结构

第一章绪论，从教育信息化建设现实需求出发，阐明本研究所要解决的核心问题，通过国内外文献调研指出现有研究的局限性，提出本文的研究方向，概述各章节主要内容。

第二章相关技术介绍，深入介绍与系统开发相关的核心技术，包括 Spring Cloud Alibaba 微服务框架、Nacos 服务治理、Redis 缓存与 Redisson 分布式锁、RabbitMQ 消息队列、Sa-Token 鉴权、MyBatis-Plus 数据访问以及 React 前端技术栈，并详细说明系统开发环境配置。

第三章系统分析，从技术可行性、用户操作可行性和开发成本可行性三个维度论证系统建设的可行性，采用用例驱动方法梳理三类用户的功能需求，形成完整的需求规格说明。

第四章系统设计，阐述系统的整体四层架构设计，详细规划各功能模块的设计方案，完成数据库概念设计（E-R 图）和逻辑设计（七张核心表结构定义），为系统实现提供完整蓝图。

第五章系统实现，以模块为单元通过实现流程说明与界面截图相结合的方式，系统性呈现六大模块的实现细节，验证各模块技术方案的可行性与功能完整性。

第六章系统测试，设计覆盖六大功能模块的测试用例，通过功能测试验证业务逻辑正确性，借助 JMeter 阶梯式性能压力测试评估系统高并发处理能力。

第七章总结与展望，总结微服务架构在智能选课系统中的实践效果，并就推荐算法深化、弹性伸缩增强、移动端开发等方面提出未来演进方向。

## 1.5 本章小结

本章系统阐述了选课系统智能化改造的研究背景与重要意义，通过国内外文献调研梳理了现有研究成果的核心不足，提出了本论文在微服务架构下的六大核心模块设计思路，并清晰勾勒了全文的章节结构与论述脉络，为后续各章节的深

入展开奠定了基础。

# 第 2 章 相关技术介绍

## 2.1 相关技术

### 2.1.1 Spring Boot 与 Spring Cloud Alibaba 微服务架构

Spring Boot 由 Pivotal 团队提出，其核心理念是"约定优于配置"。借助自动装配机制，传统开发中大量依赖 XML 的配置工作得到显著简化。本系统基于 Spring Boot 2.6.13 版本开发，借助@SpringBootApplication 注解完成配置类、自动配置与包扫描的统一组织，并通过内嵌 Tomcat 实现快速启动与部署[5]。得益于丰富的 starter 体系，项目能够以较低的配置成本完成核心功能集成。

Spring Cloud Alibaba 是阿里巴巴基于 Spring Cloud 规范构建的扩展实现，能够将 Nacos、Sentinel 等国内常用中间件高效整合进微服务体系。本系统采用 Spring Cloud Alibaba 搭建微服务治理框架，具体包括通过 Nacos 完成服务发现、通过 OpenFeign 实现跨服务调用、通过 Nacos Config 集中管理公共配置等。相较于传统的 Eureka 与 Config Server 组合，该方案在当前项目场景下具有更高的集成效率与配置便利性。

### 2.1.2 Nacos 服务注册与配置中心

Nacos 全称 Dynamic Naming and Configuration Service，是阿里巴巴开源的一款服务发现与配置中心，在国内微服务项目中应用较为广泛[6]。在本系统中，Nacos 主要承担两项职责：其一，作为服务注册中心，各微服务启动后将自身地址与端口信息注册至 Nacos，Spring Cloud Gateway 据此完成动态路由，无需手工维护服务地址；其二，作为配置中心，数据库连接、Redis 地址等公共配置统一存放于 Nacos，各服务启动时按需拉取，结合@RefreshScope 注解后可支持配置热更新。

### 2.1.3 Redis 缓存与 Redisson 分布式锁

Redis（Remote Dictionary Server）是一款基于内存的高性能键值存储系统，支持 String、Hash、List、Set、Sorted Set 等丰富数据结构，以亚毫秒级响应延迟著称。在本系统中，Redis 主要承担两项职责：一是作为 Sa-Token 与相关会话组件的后端存储，实现多服务实例间的登录态共享；二是作为 Redisson 分布式锁的底层协调介质，为选课临界区并发控制提供支撑[6]。

Redisson 是构建在 Redis 之上的 Java 分布式对象与服务框架，提供了较为丰富的并发控制能力。本系统在选课核心流程中利用其分布式锁能力对课程维度的关键操作进行串行化控制，并设置锁等待时间与租约时间，以确保同一课程在同一时刻仅有一个请求进入库存扣减临界区，从而降低并发超卖风险。

### 2.1.4 RabbitMQ 消息队列与异步处理

RabbitMQ 是基于 AMQP 协议的开源消息代理，以可靠投递、灵活的交换机路

由和完善的管理界面而广泛应用于分布式系统的异步解耦场景[7][8]。本系统引入 RabbitMQ 实现双重目标：其一，流量削峰，主业务服务将选课请求封装为 EnrollmentDTO 消息发送至队列后立即返回"处理中"响应，选课微服务从队列中按序消费并执行实际的选课逻辑，将峰值并发压力从数据库层面平移至消息队列层面；其二，异步通知，选课结果（成功/失败及原因）通过通知队列异步发送至站内消息中心，实现了选课处理与通知推送的彻底解耦，确保通知的可靠投递。

### 2.1.5 Sa-Token 统一鉴权框架

Sa-Token 是一个轻量级 Java 权限认证框架，以简洁的 API 设计和开箱即用的功能特性著称，可较为便捷地完成登录认证与权限校验[9]。在微服务场景下，统一鉴权与安全治理还需关注服务边界上的潜在安全异味和风险点[12]。本系统通过 Sa-Token 实现统一鉴权：用户登录成功后由框架生成会话 Token 并结合 Redis 进行存储；网关层通过自定义鉴权过滤器拦截请求，对登录、注册等白名单接口放行，其余接口统一执行登录态校验。当前配置中 Token 有效期为 30 天，active-timeout 为-1，整体策略更偏向于保证开发和演示阶段的登录连续性。

### 2.1.6 MyBatis-Plus 数据访问框架

MyBatis-Plus 是在 MyBatis 基础上进行功能增强的 ORM 框架，提供了通用 Mapper、通用 Service、条件构造器、逻辑删除与自动填充等特性，可显著减少单表 CRUD 操作中的样板代码。本系统全面采用 MyBatis-Plus 进行数据访问，通过逻辑删除机制管理 is_delete 字段，并利用自动填充能力维护 create_time 与 update_time 字段，从而规范化数据操作行为并提升开发效率。数据库结构设计与 E-R 建模部分同时参考了近年的 MySQL 数据库设计教材与工程实践资料[14]。

### 2.1.7 React 与 Ant Design Pro 前端框架

React 是由 Meta 开发的声明式、组件化 JavaScript 视图库，通过虚拟 DOM 差分算法实现高效的 UI 局部更新，结合 Hooks 特性实现函数式组件状态管理，是当前最主流的前端框架之一[10]。Ant Design Pro 是基于 Ant Design 企业级设计体系的前端应用框架，提供了布局方案、权限管理等开箱即用功能，内置 ProTable（高级表格）、ProForm（高级表单）等业务组件，可大幅缩短管理后台的开发周期。本系统前端采用 React + TypeScript + Ant Design Pro 技术栈，通过 UmiJS 框架管理路由，使用 Axios 封装 HTTP 请求，系统界面自适应 PC 与移动端多种分辨率。

## 2.2 开发环境

### 2.2.1 开发工具

集成开发环境：后端开发采用 IntelliJ IDEA 2023 Ultimate 版，通过深度集成的 Spring Boot 插件、Nacos 工具、Docker 支持，实现从微服务编码到容器化构建的全流程开发。前端开发采用 VS Code，配合 ESLint、TypeScript 语言服务和 Ant

Design Pro 代码片段，提升界面开发效率与代码质量。版本管理采用 Git，通过 Gitee 托管代码仓库，执行特性分支开发模式，commit message 遵循约定式规范。数据库管理方面，Navicat Premium 15 用于 MySQL 管理，Another Redis Desktop Manager 用于 Redis 监控。接口联调与功能测试阶段使用 Postman 对登录、推荐、选课、退课、消息查询等核心接口逐项发送请求并核对响应结果，为第六章测试过程整理提供原始记录。

### 2.2.2 运行环境

后端按 pom 配置使用 Java 8，构建工具是 Maven 3.9。中间件方面，MySQL 8.0 存储业务数据，Redis 用于会话与分布式锁支撑，RabbitMQ 负责消息队列并开启了消息持久化配置，Nacos 2.x 作为注册与配置中心。前端基于 Node.js 运行环境，用 npm 管理依赖，锁定 package-lock.json 来保证开发和生产环境的一致性。UmiJS 内置 Webpack 做打包，支持按路由懒加载。安全方面，部署阶段可由 Nginx 承担 HTTPS 终止，日志通过 SLF4J + Logback 输出，方便排查问题。

## 2.3 本章小结

本章系统梳理了选课系统开发所依赖的核心技术体系，从微服务框架（Spring Boot + Spring Cloud Alibaba）、服务治理（Nacos）、缓存与分布式锁（Redis + Redisson）、消息队列（RabbitMQ）、安全认证（Sa-Token）、数据访问（MyBatis-Plus）到前端技术栈（React + Ant Design Pro）进行了全面深入的介绍。这些技术的有机组合为本系统高可用、高并发、易扩展的目标提供了坚实的技术基础，为后续系统设计与实现奠定了理论依据。

# 第 3 章 系统分析

## 3.1 可行性分析

### 3.1.1 技术可行性

本系统所采用的技术组件均已在大量生产环境中得到充分验证，技术成熟度高，社区活跃，文档完善，具备充分的技术可行性。在高并发防护方面，Redisson 分布式锁已被广泛用于秒杀、抢购等高并发场景，是解决库存超卖问题的成熟方案；RabbitMQ 用于将瞬时请求压力从同步链路转移到异步处理链路；Spring Cloud Gateway 负责统一接入和登录态校验，三者组合能够满足本系统的工程实现需求。

在推荐算法的可行性方面，本文有意避开机器学习方向。原因在于高校服务器资源相对有限，训练数据规模也较为有限，若直接引入复杂模型，收益与成本并不匹配。最终采用基于专业字段的规则匹配与评分方法，算法复杂度为 O(n log n)，计算过程无需额外模型服务支撑，可直接在 Java 服务中完成。冲突检测方面，区间交叉判断属于基础算法，一个学生的已选课程数量通常有限，逐门遍历即可满足业务需要。总体来看，这两类功能对现有软硬件环境要求较低，具备较强的技术可行性。

### 3.1.2 用户操作体验可行性

系统采用 Ant Design Pro 的响应式栅格布局，在 PC、平板、手机三类终端设备上均能提供一致的操作体验，核心功能适配率达 100%。学生端通过"一键选课"设计将传统多步骤流程简化为"筛选课程→点击选课→弹窗确认"三步操作，并在确认弹窗中集成冲突检测结果展示，帮助学生提交前充分了解潜在风险。个性化推荐列表以卡片形式呈现，清晰标注推荐理由，有效降低学生的选课决策成本。管理员端关键操作控制在 3 步以内，系统响应时间控制在 1 秒内，符合教育部相关信息化系统可用性标准。

### 3.1.3 开发成本可行性

本系统由笔者独立开发完成，所采用技术栈均为开源方案，软件授权成本较低。开发工具方面，后端使用学校提供教育授权的 IntelliJ IDEA，前端使用 VS Code。部署资源方面，系统运行于一台 2 核 4G 基础云服务器，中间件通过 Docker Compose 统一编排。综合服务器、域名及其他支出，全年总成本控制在 300 元以内，符合毕业设计项目对开发成本的预期要求。

从工作量构成看，项目不仅完成了需求分析、数据库设计、微服务拆分、前后端编码与接口联调，还覆盖了容器化部署、功能测试、性能测试、截图取证、论文撰写与反复修改等完整工程链路，整体工作量能够满足软件工程专业本科毕业设计的基本要求。

## 3.2 功能需求分析

### 3.2.1 功能描述

系统面向三类用户，不同角色的功能需求各有侧重。学生是系统的高频使用者，其核心诉求包括快速定位目标课程、在提交前获知潜在时间冲突以及在选课完成后及时接收结果通知。具体功能包括：按课程名称、教师、授课时间、专业方向等多个条件筛选课程；查看系统基于专业生成的推荐列表及推荐理由；在确认选课前弹出时间冲突检测结果；查看个人课程表；接收选课成功或失败的站内通知；以及执行退课操作。系统设计还需满足较高并发访问场景下的稳定运行要求。

教师用户核心需求：课程信息发布与维护（填写课程名称、学分、授课时间、地点、适用专业等属性后提交审核）；课程信息修改（修改后需重新触发审核）；选课学生名单查看与数据导出；接收课程审核通过/驳回通知。管理员用户核心需求：学生与教师信息全生命周期管理（增删改查）；课程信息审核（通过/驳回，可附驳回原因）与强制扩容；未读消息管理。

### 3.2.2 管理员用例图

管理员在系统里主要扮演运维和审核的角色，权限最高。他需要能管理教师和学生账户，处理课程审核请求（通过或者驳回，驳回时要给出理由），必要时对名额不足的课程扩容，并维护系统基础数据。管理员用例图如图 3.1 所示。

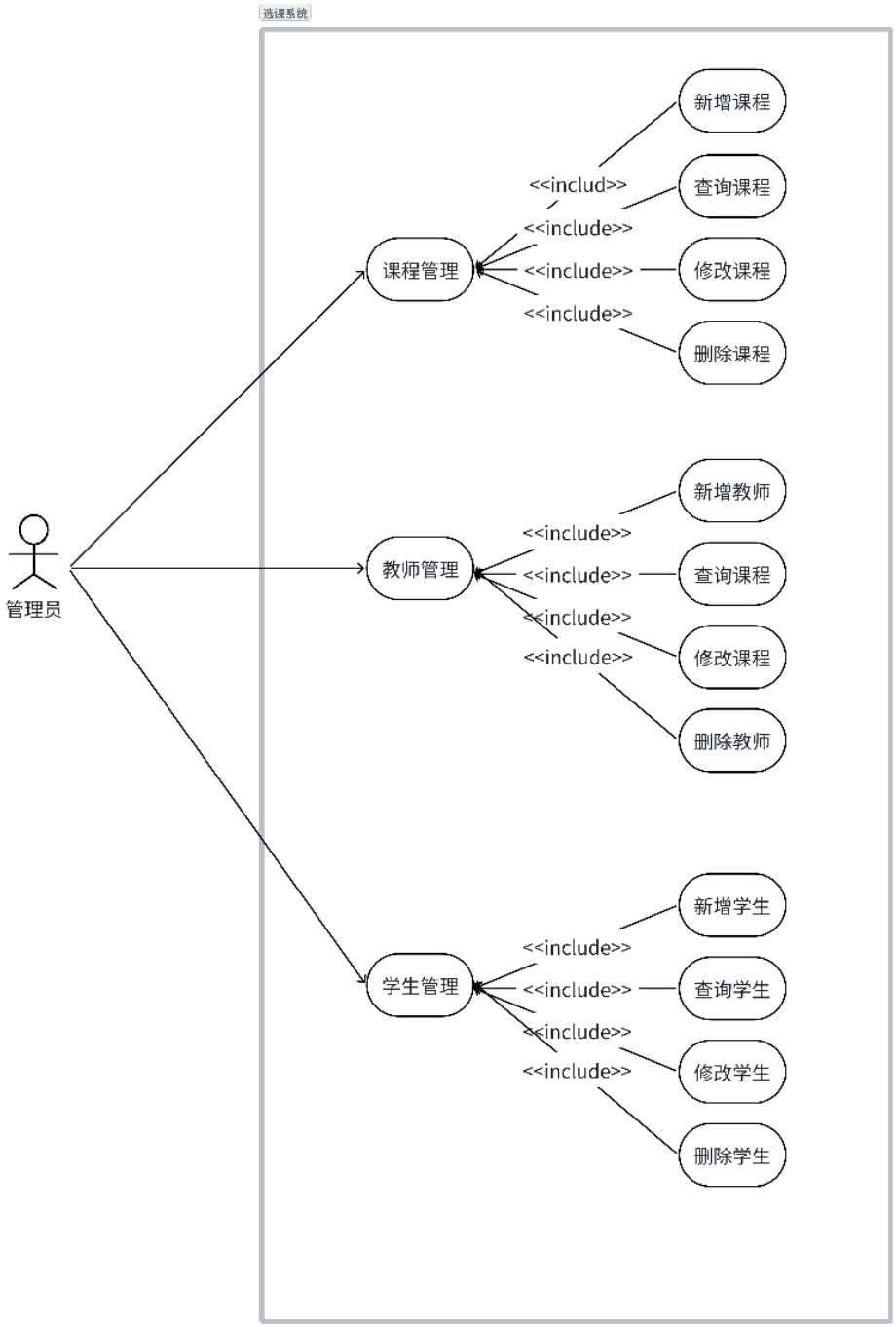



图 3.1 管理员用例图

### 3.2.3 用户综合用例图

学生角色完成选课全流程：通过多条件筛选定位目标课程，系统在提交选课前自动执行时间冲突检测并以弹窗展示检测结果，学生确认无冲突后提交，系统异步处理后通过消息中心推送选课结果；课后可通过课表页面管理已选课程。教师角色聚焦课程管理，提交审核申请，待管理员审核通过后课程上线供学生选择。用户综合用例图如图 3.2 所示。

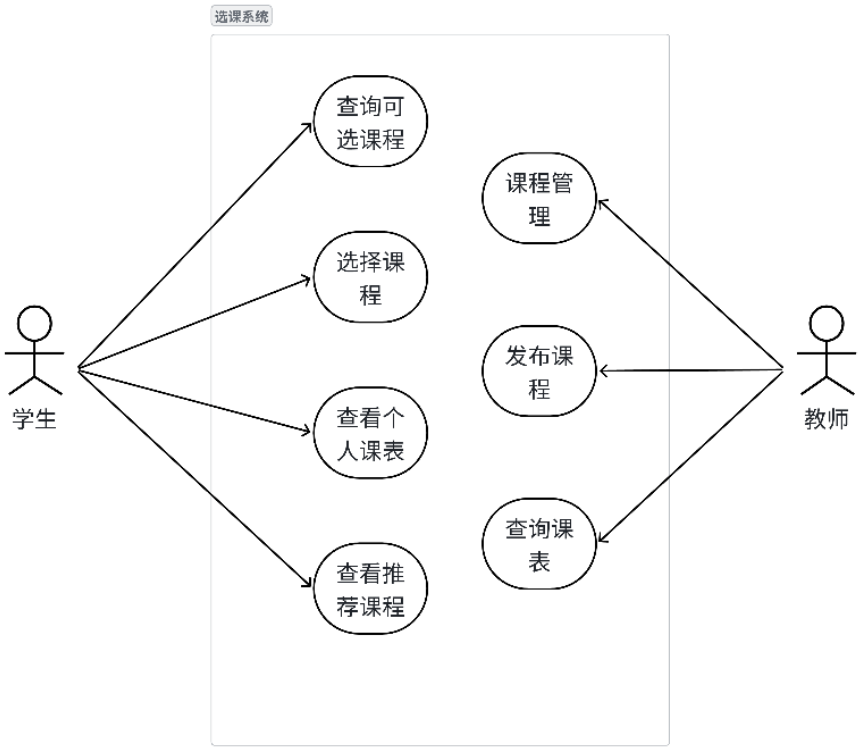



图 3.2 用户综合用例图

## 3.3 本章小结

本章从技术可行性、用户体验和开发成本三个角度论证了系统建设的合理性，所有选用的技术组件都有成熟的社区支撑，个人开发成本也控制在合理范围内。需求方面，以用例图的形式梳理了三类用户的业务诉求，核心功能边界已经明确，为后续的设计和开发工作提供了依据。

# 第 4 章 系统设计

## 4.1 系统架构设计

系统整体采用四层架构设计，各层职责边界明确。

最外层是访问层，三类用户通过浏览器发起 HTTPS 请求，前端打包后的静态文件由 Nginx 托管，所有通信强制走 TLS 加密，防止数据在传输过程中被截取。

接入层有两个角色：Nginx 在前负责静态资源分发和隐藏后端真实 IP；Spring Cloud Gateway 在后充当微服务网关，通过 Nacos 服务发现动态路由到各微服务实例。所有请求在进入业务服务前先经过 Sa-Token 过滤器做登录态校验，并统一处理跨域与路由转发。

应用层：主业务服务（main-service，端口 8101）处理低频业务，包括用户管理、课程管理、智能推荐、冲突检测等；选课微服务（select-service，端口 8081）专注处理高频高并发的选课业务，通过 Redisson 分布式锁与 RabbitMQ 保障数据一致性。两个服务均向 Nacos 注册，通过 OpenFeign 实现跨服务远程调用。

数据存储层：MySQL 8.0 提供可靠的关系型业务数据持久化存储；Redis 6.0 提供高速内存缓存与分布式会话存储；RabbitMQ 承担异步消息传递与流量削峰职责，三者协同形成高性能、高可靠的数据支撑体系。结合微服务拆分与独立部署的工程实践，系统总体架构保持了较好的可扩展性[13]。

## 4.2 总体结构设计

按照 2026 版毕业论文写作要求，系统功能结构图应与需求分析、详细设计和实现模块保持一致。因此，本文将系统划分为学生端、教师端、管理员端和公共支撑四个一级模块，各一级模块下再细化出课程查询、智能推荐、在线选课、课程审核、消息通知等核心子模块。系统功能结构图如图 4.1 所示。

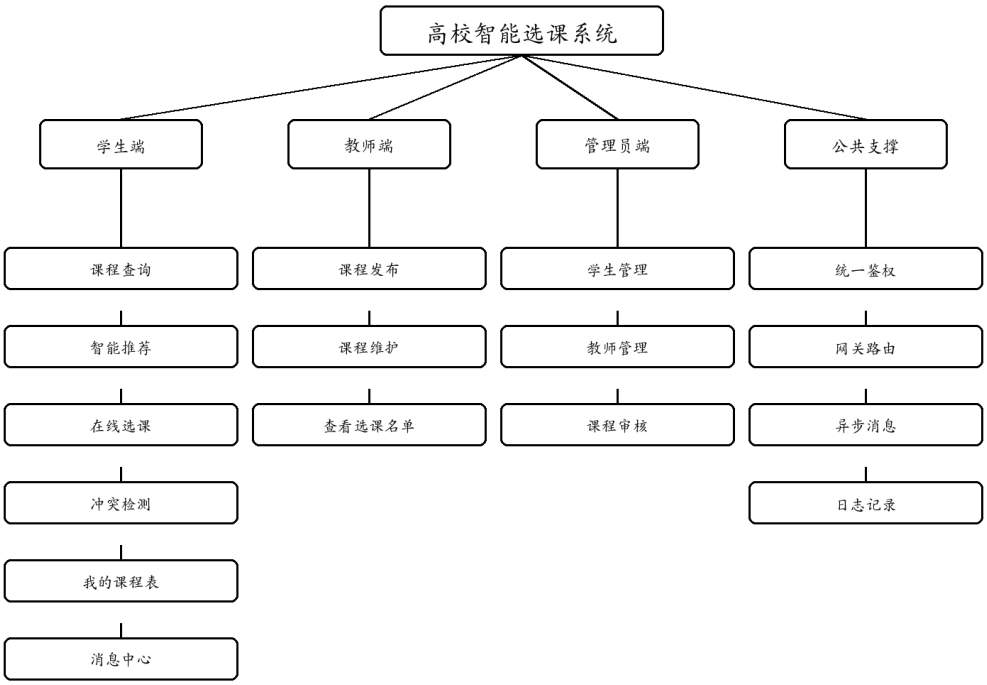



图 4.1 系统功能结构图

### 4.2.1 用户服务层

三类用户的功能入口与界面设计具有明显差异。学生端提供课程查询、推荐课程、选课和课程表四个主要功能模块，选课结果通过消息中心通知，并在提交前展示冲突检测结果。教师端主要承担课程发布与维护工作，提交后进入审核流程，同时可查看选课名单。管理员端权限最高，负责账户管理、课程审核及基础运维等管理工作。

### 4.2.2 核心业务层

（1）智能流量调控体系：Spring Cloud Gateway 统一入口，RabbitMQ 消息队列承担请求削峰，Redisson 分布式锁负责选课临界区并发控制。三者组合将高峰期的同步压力转移到更可控的异步处理链路中。

（2）智能选课引擎：包含三个核心子功能：个性化推荐（专业匹配评分，高相关性课程优先）、冲突预检（提交前自动遍历已选课程进行时间区间交叉比对）、库存管理（课程库存字段配合分布式锁控制名额扣减）。

（3）综合安全保障体系：Sa-Token 与 Spring Cloud Gateway 负责统一登录校验，用户角色字段用于区分管理员、教师、学生三类账户，Nginx 通过 SSL/TLS 加密防止数据传输过程中被窃听或篡改。

### 4.2.3 数据支撑层

数据存储体系：业务数据库存储课程、选课记录、用户档案等核心数据；选

课日志表记录每次异步处理对应的 requestId 和状态，用于结果回查与问题溯源。数据一致性机制：选课成功后即时更新课程库存与选课记录；消息通知落库后同步更新未读消息计数。数据分析服务：选课热点统计识别高频访问课程，为资源分配提供数据依据；选课成功率统计用于系统性能优化参考。

## 4.3 功能模块设计

### 4.3.1 智能课程推荐模块

推荐模块以学生专业为匹配维度，通过 OpenFeign 调用选课微服务获取课程分页数据，对专业对口课程赋值 90 分，对通用选修课程赋值 50 分，再按得分排序后返回前 N 条结果。若专业匹配课程数量不足设定的推荐数，则以通用课程补足。每条推荐结果均附带推荐理由，例如"该课程与你的专业相关"或"通用选修课程"。系统默认返回 5 条推荐结果，前端可通过 limit 参数进行调整。

### 4.3.2 选课模块

选课模块需要重点解决高并发场景下的数据一致性问题。其处理流程如下：主业务服务先将选课请求投递至 RabbitMQ 队列并立即返回"处理中"提示；异步消费者为本次处理生成 requestId 后调用选课微服务；选课微服务内部先查询 enrollment_log 中的同 requestId 记录，再尝试获取 Redisson 分布式锁，等待上限 500ms，持锁最长 5000ms；加锁成功后继续校验库存是否大于 0，以及该学生是否已经选过该课程；两项校验均通过后写入选课记录并扣减库存；若抢锁失败则返回"系统繁忙，请稍后重试"。整个流程完成后，选课结果通过 RabbitMQ 异步发送通知。选课模块时序图如图 4.2 所示。

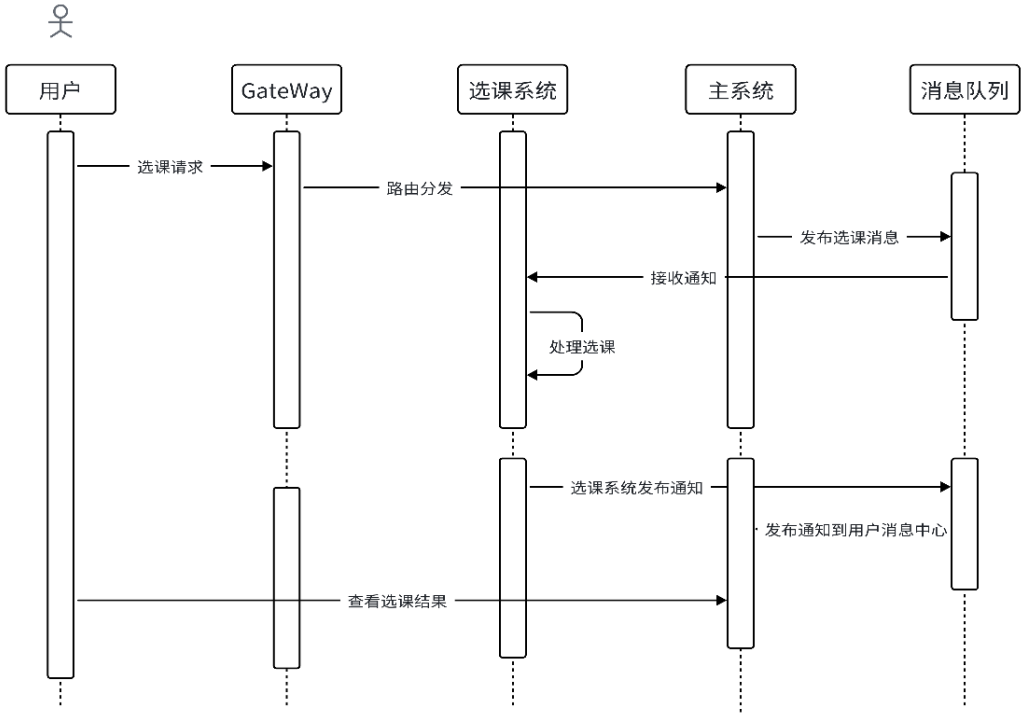


图 4.2 选课模块时序图

### 4.3.3 课程冲突检测模块

冲突检测的核心是时间窗口扩展算法。考虑到学生在不同教学楼之间转场通常需要 10 至 15 分钟，若仅采用精确时间点匹配，可能遗漏"形式上不冲突、实际无法及时到达"的情况。因此，系统将目标课程的开始时间提前 15 分钟、结束时间延后 15 分钟，形成扩展区间，再与已选课程逐一进行交叉比对。交叉判断覆盖三种情形：已选课程的开始时间落在扩展区间内、结束时间落在扩展区间内，或完全包含扩展区间。检测到冲突后返回冲突课程列表和原因说明，用于对主要时间重叠场景进行预警。

### 4.3.4 系统管理模块

系统管理模块按角色划分为三层权限：管理员具备完整管理权限；教师仅可维护本人创建的课程；学生仅保留查询类权限。课程上线遵循明确的审批流程：教师创建课程并提交后，课程处于"待审核"状态（isCheck=0），此时不对学生展示；管理员审核通过后状态更新为 1，课程方可进入学生查询列表；若审核驳回则状态更新为 2，并可填写驳回原因，教师根据意见修改后可再次提交。

### 4.3.5 消息通知与网关安全模块

消息通知模块采用 RabbitMQ 实现通知的异步可靠发送。选课处理完成后，系统向通知队列发送消息，消费者接收后写入 message 表，同时对用户的未读消息数字段执行加一操作。前端采用定时轮询方式每隔 30 秒查询一次未读数，若存在新消息，则在导航栏显示红点提示。网关安全方面，Spring Cloud Gateway 负责统一入口鉴权与跨域处理，Sa-Token 负责登录态校验，Nginx 配合 SSL 证书保障传输安全[15]。

## 4.4 数据库设计

### 4.4.1 概念设计

系统涉及的核心数据实体有七类：用户、学生、教师、课程（Subject）、课程安排（Curriculum）、选课记录（Enrollment）、选课日志（EnrollmentLog）。课程安排和学生之间是多对多关系，通过选课记录表来桥接。选课日志单独建表，记录每次选课操作的请求 ID 和处理状态，用于结果回查与异常排查。系统 E-R 图如图 4.3 所示。

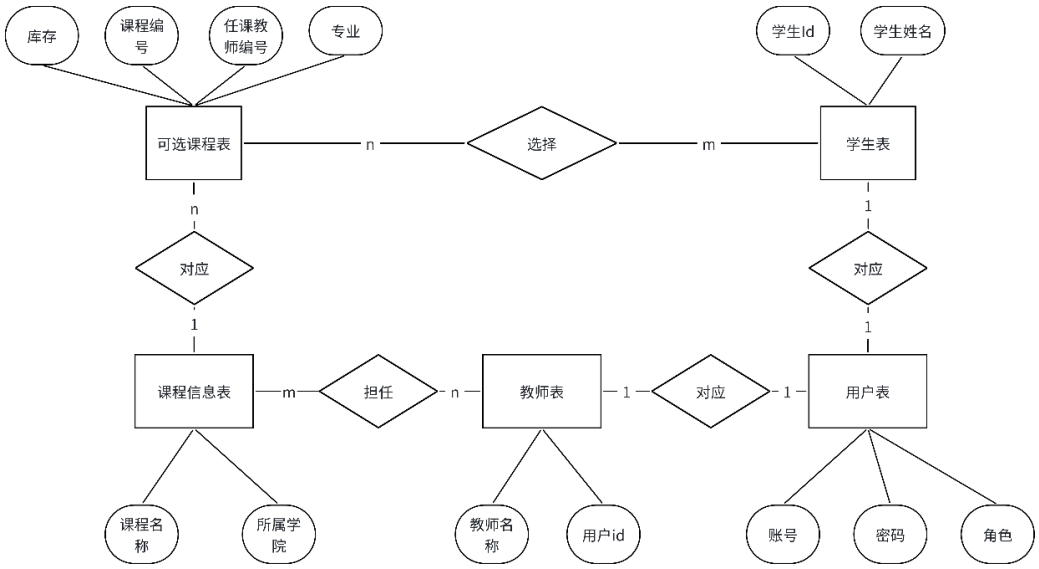



图 4.3 系统 E-R 图

### 4.4.2 逻辑设计

（1）用户表（user）—— 存储所有用户的账户信息与基础属性，通过 userRole 字段区分管理员、教师、学生三类角色。该表同时保存个性签名、头像、性别以及未读消息计数等通用信息。

表 4.1 用户表（user）

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | bigint | - | 是 |
| userName | 用户昵称 | varchar | 256 | 否 |
| userAccount | 登录账号 | varchar | 256 | 否 |
| userPassword | 登录密码 | varchar | 512 | 否 |
| userDescription | 个性签名 | varchar | 255 | 否 |
| userAvatar | 用户头像 URL | varchar | 1024 | 否 |
| gender | 性别 | tinyint | - | 否 |
| userRole | 用户角色 | varchar | 256 | 否 |
| createTime | 创建时间 | datetime | - | 否 |
| updateTime | 更新时间 | datetime | - | 否 |
| isDelete | 逻辑删除 | tinyint | - | 否 |
| unReadMessage | 未读消息数 | int | - | 否 |

（2）学生表（student）——存储学生特有信息，通过 userId 与用户表关联。除学号、姓名外，还保存年级、学院、联系方式等信息，其中 major 字段直接参与推荐算法匹配。

表 4.2 学生表（student）

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| studentId | 学号 | bigint | - | 否 |
| userId | 关联用户 ID | bigint | - | 否 |
| studentName | 学生姓名 | varchar | 50 | 否 |
| gender | 性别 | varchar | 10 | 否 |
| age | 年龄 | int | - | 否 |
| grade | 年级 | varchar | 20 | 否 |
| major | 专业 | varchar | 50 | 否 |
| college | 学院 | varchar | 50 | 否 |
| phoneNumber | 联系电话 | varchar | 20 | 否 |
| email | 邮箱 | varchar | 50 | 否 |
| createTime | 创建时间 | datetime | - | 否 |
| updateTime | 更新时间 | datetime | - | 否 |

（3）教师表（teacher）——存储教师基本信息。

表 4.3 教师表（teacher）

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| teacherId | 工号 | bigint | - | 否 |
| userId | 关联用户 ID | bigint | - | 否 |
| teacherName | 教师姓名 | varchar | 50 | 否 |
| level | 职称 | varchar | 20 | 否 |

（4）课程表（subject）——存储课程基本信息，课程名称、开课学院和学分均在该表中维护。

表 4.4 课程表（subject）

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| subjectName | 课程名称 | varchar | 50 | 否 |
| college | 开课学院 | varchar | 50 | 否 |
| credit | 课程学分 | double | - | 否 |
| createTime | 创建时间 | datetime | - | 否 |
| updateTime | 更新时间 | datetime | - | 否 |

（5）课程安排表（curriculum）——存储具体课程安排。该表除授课时间、地点、适用专业外，还维护星期、节次、审核状态以及总名额和剩余名额，用于支撑冲突检测、审核流与库存控制。

表 4.5 课程安排表（curriculum）

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| subjectId | 课程 ID | int | - | 否 |

| | | | | |
|---|---|---|---|---|
| teacherId | 教师 ID | int | - | 否 |
| teachingTime | 授课时间 | datetime | - | 否 |
| dayOfWeek | 星期几(1-7) | int | - | 否 |
| startTime | 开始节次 | int | - | 否 |
| endTime | 结束节次 | int | - | 否 |
| location | 授课地点 | varchar | 50 | 否 |
| remarks | 课程备注 | varchar | 255 | 否 |
| grade | 适用年级 | varchar | 20 | 否 |
| major | 适用专业 | varchar | 50 | 否 |
| isCheck | 审核状态(0 待/1 过/2 驳) | tinyint | - | 否 |
| stock | 剩余名额 | int | - | 否 |
| isStock | 是否限额 | int | - | 否 |
| createTime | 创建时间 | datetime | - | 否 |
| updateTime | 更新时间 | datetime | - | 否 |
| isDelete | 逻辑删除 | tinyint | - | 否 |
| allStock | 课程总名额 | int | - | 否 |

（6）选课记录表（enrollment）——记录学生与课程安排的多对多关联关系。

表 4.6 选课记录表（enrollment）

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | bigint | - | 是 |
| studentId | 学生 ID | bigint | - | 否 |
| curriculumId | 课程安排 ID | bigint | - | 否 |
| createTime | 创建时间 | datetime | - | 否 |
| updateTime | 更新时间 | datetime | - | 否 |

（7）选课日志表（enrollment_log）——记录所有选课操作历史，用于处理状态记录与问题追溯。

表 4.7 选课日志表（enrollment_log）

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | bigint | - | 是 |
| requestId | 请求处理 ID | bigint | - | 否 |
| studentId | 学生 ID | bigint | - | 否 |
| status | 操作状态(0 处理中/1 成功/2 失败) | tinyint | - | 否 |
| createTime | 创建时间 | datetime | - | 否 |
| updateTime | 更新时间 | datetime | - | 否 |

## 4.5 本章小结

本章从四层系统架构、三大业务体系（流量调控/选课引擎/安全保障）到七张数据库表结构，完整呈现了系统的概要设计方案。四层架构设计通过关注点分离

实现了安全防护、性能优化与弹性扩展的有机统一；功能模块设计充分考虑了高校选课业务的特殊需求；规范化的数据库表结构为系统功能实现提供了可靠的数据基础，为后续详细设计与编码实现提供了清晰蓝图。

# 第 5 章 系统实现

## 5.1 智能课程推荐模块

### 5.1.1 推荐算法设计与实现

智能推荐的核心逻辑位于推荐服务层中。系统首先根据 studentId 查询学生专业信息，再通过 OpenFeign 调用选课微服务的分页接口获取按专业过滤的课程列表，并赋予 90 分；若专业匹配课程数量不足预设 limit，则补充通用课程并赋予 50 分；最后对所有候选课程按得分降序排列，截取前 limit 条返回。主服务不单独维护课程数据副本，课程信息均由选课微服务实时提供，以保证数据一致性。其实现过程可概括为以下三个步骤：首先校验 studentId 与 limit 参数，若学生不存在则直接返回空列表；其次依据学生专业查询高相关课程，不足部分再以通用选修课程补足；最后按照推荐分值完成排序与截取，并封装推荐理由后返回前端。

推荐流程通过 Feign 完成跨服务调用，整体逻辑在 Java 服务侧执行，不依赖外部模型服务。排序阶段采用比较器完成降序处理，算法复杂度为 O(n log n)。高校场景下课程总量通常有限，这类基于规则的推荐方式能够在较低计算开销下给出稳定结果，也便于后续继续扩展更多推荐维度。

### 5.1.2 推荐课程页面

推荐页面采用卡片网格布局展示结果，每张卡片显示课程名称、授课时间、授课地点、适用专业及推荐理由，右下角设置"立即选课"按钮以便直接进入选课流程。页面加载完成后自动请求推荐接口，在 loading 阶段展示 Ant Design 的 Spin 组件，数据返回后渲染卡片列表。推荐课程页面如图 5.1 所示。

图 5.1 推荐课程页面

## 5.2 选课模块的实现

### 5.2.1 高并发选课核心实现

选课的核心控制逻辑位于选课微服务控制层。为保证高并发场景下的处理稳定性，系统主要依赖三道防线：requestId 处理日志记录状态、Redisson 分布式锁防止并发超卖、库存校验作为兜底机制。该控制逻辑可以概括为：首先依据 requestId 查询 enrollment_log，判断当前请求是否已有处理记录；其次基于课程 ID 尝试获取分布式锁，并在持锁状态下完成库存校验与重复选课校验；最后在条件满足时写入处理日志、调用选课服务完成入库与扣减库存，并在释放锁后将结果返回给上层调用方。

分布式锁的等待时间设置为 500ms，锁租约时间设置为 5000ms，前者用于限制抢锁等待时长，后者用于避免服务异常时长期占锁。这里的 requestId 并不是前端直接传入的业务参数，而是异步消费者在调用选课微服务前生成的内部处理标识，并同步写入 enrollment_log 用于记录本次处理状态。主业务服务通过消息发送组件将 EnrollmentDTO 投递至 RabbitMQ 队列实现异步处理，前端在收到"处理中"响应后展示等待提示，选课结果在 0 至 5 分钟内通过消息中心通知学生。

### 5.2.2 选课操作页面

进入选课页面后，学生通过顶部搜索栏输入课程名称或教师姓名进行筛选，课程列表以分页表格形式展示（每页 20 条）。点击目标课程右侧"选择课程"按钮，系统弹出确认对话框，显示课程完整信息及冲突检测结果。学生确认后点击"确认选课"，界面立即反馈"正在处理中"状态提示。选课页面如图 5.2 所示，选课操作确认弹窗如图 5.3 所示。

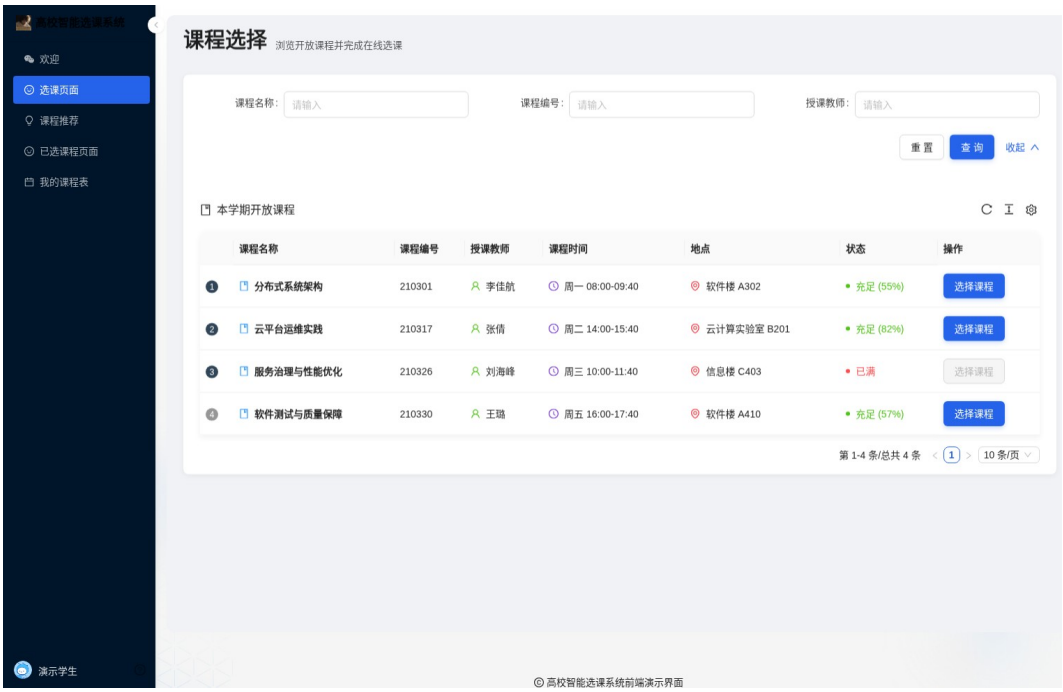



图 5.2 选课页面

图 5.3 选课操作确认弹窗

### 5.2.3 选课结果通知页面

学生的所有选课结果均通过站内消息中心统一发布。当前实现下，通知消息以结果提示和失败原因说明为主；在重复选课等部分场景中，系统还会追加课程名称，帮助学生及时判断后续是否需要调整选课方案。选课结果通知页面如图 5.4 所示。

图 5.4 选课结果通知页面

### 5.2.4 统一登录与网关转发流程

系统采用统一登录页与网关令牌校验的访问控制机制。用户首次请求受保护资源时，Gateway 检测到请求未携带 Token，会先将用户重定向到统一登录页，并在回跳地址参数中保留原始目标路径；用户提交账号密码后，主系统完成认证并生成 Token 写入 Redis，前端保存 Token 后再回跳到最初访问的业务地址；Gateway 完成 Token 校验后，才会把请求路由到具体业务服务。整个统一登录与网关转发流程如图 5.5 所示。

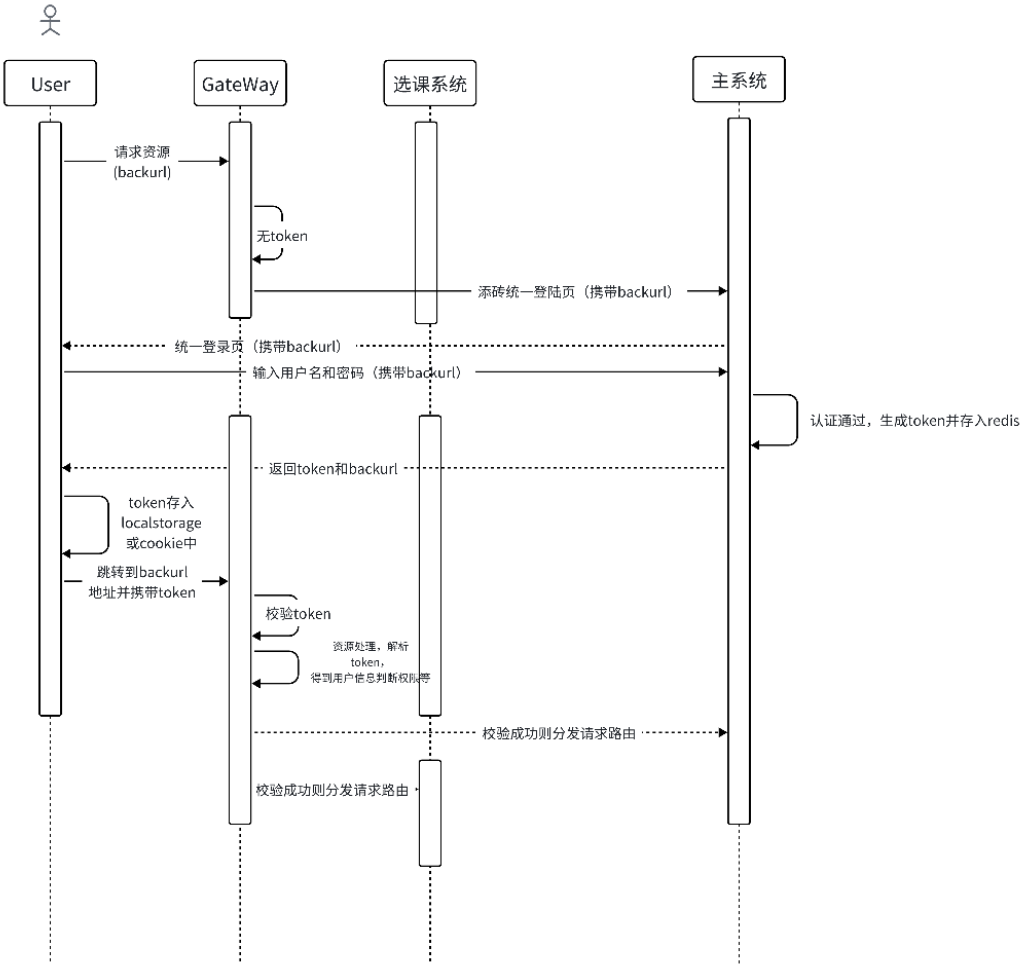



图 5.5 统一登录与网关转发流程图

## 5.3 课程冲突检测模块

### 5.3.1 冲突检测算法实现

冲突检测核心实现位于冲突检测服务的时间判断逻辑中。该逻辑通过时间区间扩展与三条件交叉判断，对课程时间重叠的主要场景进行预警。具体而言，系统首先读取两门课程的授课时间，并以 2 小时作为默认课时长度计算结束时间；随后在目标课程开始与结束时间两端各扩展 15 分钟，形成带缓冲的判断区间；最后分别从"开始时间落入区间""结束时间落入区间"以及"整体时间完全覆盖区间"三个角度判断是否构成冲突。

三个判断条件的物理含义：条件一覆盖"c2 开始时间落在 c1 扩展区间内"的场景；条件二覆盖"c2 结束时间落在 c1 扩展区间内"的场景；条件三覆盖"c2 时间完全包含 c1 扩展区间"的极端场景。三者组合能够覆盖课程时间明显重叠的主要情况。15 分钟缓冲阈值贴合高校实际情况，两节课之间若间隔不足 15 分钟，学生往往难以在不同教学楼间完成转场，因此将此类情况纳入冲突预警范围。

冲突原因描述根据时间差进行差异化生成：时间差为 0 时提示"两门课在同一

时间开始"；时间差在 15 分钟以内时提示"两门课时间相隔仅 N 分钟，可能来不及转换教室"；超出缓冲但存在区间重叠时提示"课程时间重叠"或"时间接近，可能无法及时到达"。这种解释性提示有助于学生理解冲突性质，做出更稳妥的选课决策。

## 5.4 系统管理模块的实现

### 5.4.1 学生管理页面

学生管理页面用于管理员维护学生档案，界面采用 Ant Design ProTable 展示列表，支持按学号、姓名、专业等多条件筛选，操作区提供新增、编辑、删除入口。新增学生时，系统会自动在 user 表中创建对应账户并关联 student 角色。删除操作采用逻辑删除（isDelete 置 1），不直接物理移除数据，便于后续历史查询与问题排查。学生管理页面如图 5.6 所示。

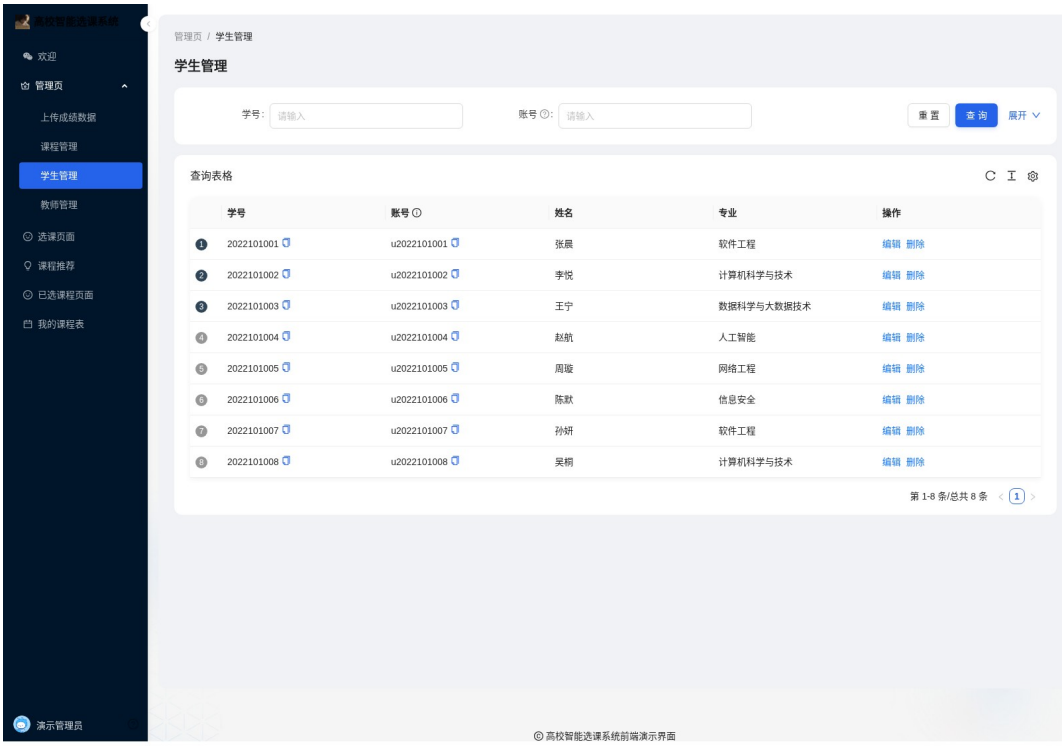



图 5.6 学生管理页面

### 5.4.2 课程管理页面

课程管理实现了二级审核流程：教师填写课程信息表单（课程名称、学分、授课时间、地点、专业方向、名额上限等字段）提交后，系统将 isCheck 置为 0（待审核），课程在审核通过（isCheck=1）前不对学生可见；管理员在课程管理页面执行通过、驳回（需填写驳回原因）操作；被驳回课程经教师修改后可重新提交审核。课程管理页面如图 5.7 所示。

图 5.7 课程管理页面

### 5.4.3 教师管理页面

教师管理页面提供教师档案的增删改查功能，字段包括工号、姓名、职称等。新增教师的处理逻辑与新增学生类似，管理员提交表单后，后台同步在 user 表中创建账户并绑定 teacher 角色，教师随后即可使用该账号登录并发布课程。教师管理页面如图 5.8 所示。

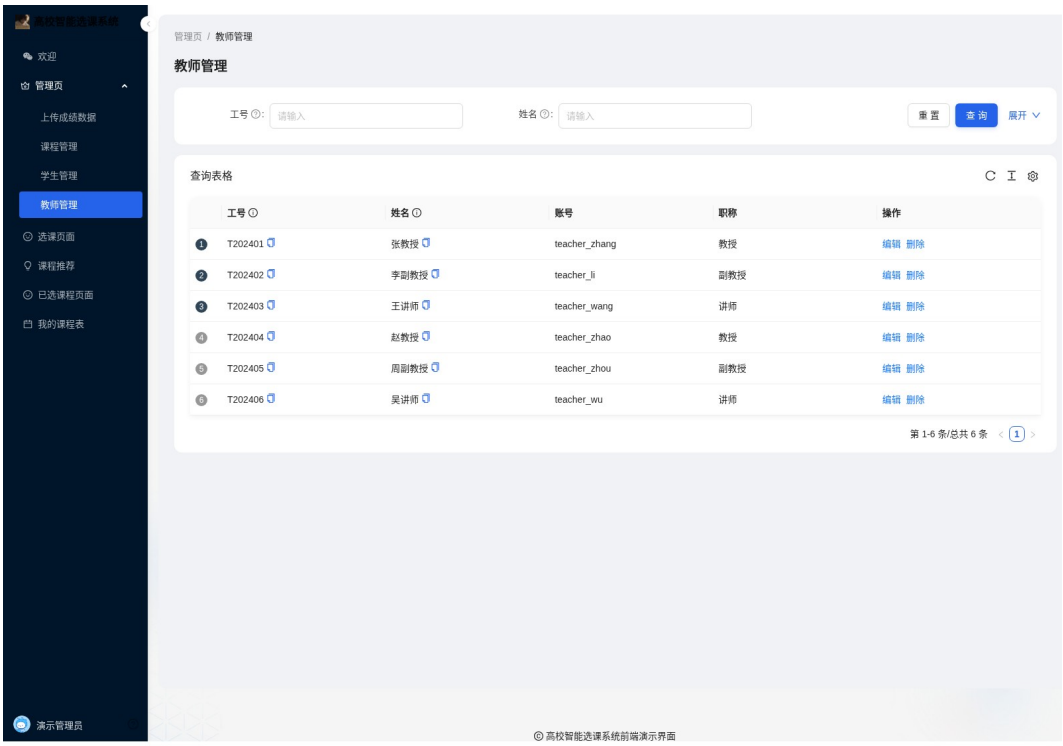



图 5.8 教师管理页面

## 5.5 消息通知模块的实现

消息通知模块采用 RabbitMQ 实现异步推送。选课处理完成后，消费者根据结果构建通知消息并发送至通知队列，通知消费者获取消息后写入 message 表，同时对用户的 unReadMessage 字段执行加 1 处理。前端采用定时轮询方式，每隔 30 秒请求一次未读数接口，若存在新消息，则在导航栏左上角显示红点数字。

当前实现下，成功通知通常展示"选课成功"等结果提示，失败通知则明确说明失败原因，比如"课程已满"或者"你已选过该课程"。在重复选课这类场景中，系统还会附带课程名称，帮助学生快速定位是哪门课触发了提示，如图 5.9 和图 5.10 所示。

图 5.9 选课成功通知示例

图 5.10 选课失败通知示例

## 5.6 系统安全模块的实现

### 5.6.1 Sa-Token 登录与 Token 管理

安全模块的鉴权工作主要由 Sa-Token 完成。用户登录时，前端将账号密码 POST 至/api/user/login，后端首先进行非空校验，随后调用 userService.userLogin()对输入密码执行加盐 MD5 处理，并与数据库中的密文进行比对；比对通过后调用 StpUtil.login(userId)创建会话，会话信息结合 Redis 进行存储，并在返回前端时一并携带用户信息与角色标识，以支持前端完成权限展示控制。登录接口的处理流程可以概括为：接收并校验登录请求参数、调用用户服务完成账号密码校验、生成 Sa-Token 会话、封装用户信息与 Token 信息后统一返回。

### 5.6.2 网关层鉴权过滤器

Spring Cloud Gateway 通过自定义鉴权过滤器拦截所有入站请求，对登录、注册以及当前预留的验证码路径直接放行，其余接口统一执行 Token 有效性验证。Sa-Token 从 HTTP 请求头中提取 Token 值，到 Redis 中查询对应会话信息，若 Token 不存在或已过期则返回错误信息。鉴权过滤器同时处理 CORS 预检请求（OPTIONS 方法），通过补充跨域响应头后直接放行，避免浏览器跨域限制影响前端正常请求。其过滤规则主要包括三部分：其一，对全部业务路径进行统一接入控制，并排除 favicon、登录、注册和验证码等白名单接口；其二，在鉴权阶段统一执行登录态校验；其三，在前置处理阶段补充跨域相关响应头，并对 OPTIONS 预检请求直接放行。

当前配置中 Token 默认有效期为 30 天，active-timeout 设置为-1，不启用无操作冻结机制，整体更偏向于保证开发与演示阶段的登录连续性。Token 结合 Redis 存储后，任意一个服务节点都可以独立完成验证，不依赖某台固定机器。通过网关鉴权后，用户即可访问受保护的课程表页面，如图 5.11 所示。

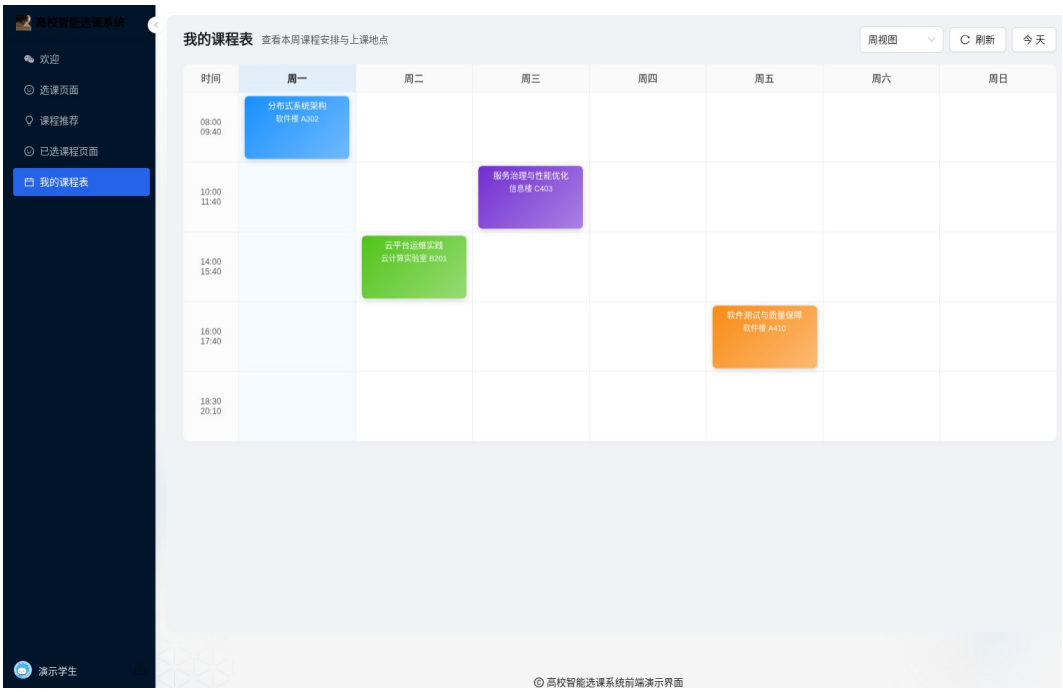



图 5.11 我的课程表页面

## 5.7 本章小结

本章以六大功能模块为核心，通过实现逻辑说明与系统界面截图相结合的方式，系统性呈现了智能推荐、高并发选课防护、时间冲突检测、系统管理、消息通知和安全鉴权模块的实现细节。智能推荐模块通过专业匹配评分实现了轻量级的个性化推荐；高并发选课模块通过 Redisson 锁、处理日志与 RabbitMQ 队列协同控制并发压力；冲突检测模块基于时间窗口扩展算法对主要重叠场景进行预警；安全模块实现了统一登录态校验与会话管理。各模块协同工作共同支撑了系统在高并发、高可靠场景下的运行。

# 第 6 章 系统测试

## 6.1 系统测试设计

系统测试用于验证系统功能完整性、性能表现以及页面展示效果是否满足设计要求。结合《毕业论文写作指导（2026 年）》中的建议，本文将测试内容划分为功能测试、性能测试和界面测试三类，并围绕系统核心模块组织测试过程。功能测试采用黑盒测试思路，重点覆盖登录鉴权、智能推荐和在线选课三个关键业务模块；性能测试聚焦高并发选课接口；界面测试主要检查登录页、推荐页和后台管理页的文字、布局与入口完整性[11]。

### 6.1.1 功能测试设计

本次功能测试采用黑盒测试方法，结合等价类划分、边界值分析和场景分析法设计测试用例。考虑篇幅控制与系统核心特征，本文重点选取登录鉴权、智能推荐和在线选课三个模块作为功能测试对象，既覆盖基础访问控制链路，也覆盖系统的特色业务逻辑。功能测试设计如表 6.1 所示。

表 6.1 核心功能测试设计表

| 用例编号 | 模块名称 | 测试说明 | 输入 | 预期结果 |
|---|---|---|---|---|
| DL001 | 登录鉴权 | 验证正确账号密码登录 | 正确账号+正确密码 | 登录成功并跳转首页 |
| DL002 | 登录鉴权 | 验证错误密码拦截 | 正确账号+错误密码 | 提示账号或密码错误 |
| TJ001 | 智能推荐 | 验证专业匹配课程优先展示 | 软件工程专业学生 | 专业相关课程优先返回 |
| TJ002 | 智能推荐 | 验证推荐数量限制 | limit=5 | 接口返回 5 条推荐数据 |
| XK001 | 在线选课 | 验证正常选课流程 | 有效 studentId 和 curriculumId | 前端收到处理中提示，最终通知成功 |
| XK002 | 在线选课 | 验证课程已满场景 | 库存为 0 的课程 | 返回选课失败且提示课程已满 |

### 6.1.2 性能测试设计

性能测试采用场景分析法设计测试方案，重点验证系统在选课高峰场景下的并发处理能力。考虑业务特点，本研究选择选课接口作为主要压测对象，并补充登录接口作为基础访问链路的对照测试。性能测试关注平均响应时间、吞吐量、错误率和库存超卖次数，性能测试设计如表 6.2 所示。

表 6.2 性能测试设计表

| 用例编号 | 测试对象 | 并发场景 | 关键指标 | 预期结果 |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| XN001 | 登录接口 | 100 并发用户 | 响应时间、错误率 | 登录链路运行稳定 |
| XN002 | 选课接口 | 100-1500 阶梯并发 | 响应时间、吞吐量、错误率、超卖次数 | 高峰场景下无超卖，主要指标处于可接受范围 |

### 6.1.3 界面测试设计

界面测试主要检查系统页面文字是否正确、模块入口是否齐全、页面排版是否存在明显错位，以及不同角色页面之间的功能边界是否清晰。结合论文展示内容，本文选取登录页、推荐页和后台管理页作为界面测试对象，界面测试设计如表 6.3 所示。

表 6.3 界面测试设计表

| 用例编号 | 测试页面 | 检查内容 | 预期结果 |
|---|---|---|---|
| UI001 | 登录页 | 系统名称、输入框、登录按钮显示 | 页面文字正确，入口完整 |
| UI002 | 推荐页 | 课程卡片、推荐理由、选课按钮显示 | 页面布局正常，核心信息清晰 |
| UI003 | 管理页 | 列表表头、查询条件、操作按钮显示 | 页面无明显错位，功能入口齐全 |

## 6.2 系统测试执行

### 6.2.1 功能测试执行

功能测试执行阶段使用 Postman 对核心接口进行请求验证，再结合前端页面操作结果进行交叉核对。测试过程中重点关注登录态校验、推荐结果合理性和选课处理结果一致性。以登录接口为例，测试过程需要同时记录请求地址、请求体、响应状态码和关键返回字段，接口测试结果如图 6.1 所示。

图 6.1 Postman 登录接口测试结果图

表 6.4 核心功能测试执行结果表

| 模块名称 | 设计用例数 | 执行用例数 | 通过数 | 未通过用例 |
|---|---|---|---|---|
| 登录鉴权 | 2 | 2 | 2 | 无 |
| 智能推荐 | 2 | 2 | 2 | 无 |
| 在线选课 | 2 | 2 | 2 | 无 |

表 6.5 功能测试问题记录表

| 问题编号 | 问题描述 | 影响模块 | 处理结果 |
|---|---|---|---|
| Bug-F01 | 早期版本中登录页标题文字与论文题目不一致 | 登录页 | 已统一修正为论文题目对应系统名称 |
| Bug-F02 | 重复选课场景下提示信息不够明确 | 选课模块 | 已补充失败原因说明与消息通知内容 |

### 6.2.2 性能测试执行

性能测试在笔者本地开发环境中进行，测试机器配置为 AMD Ryzen 7 4800U 处理器、16GB 内存，操作系统为 Linux。系统采用单机部署方式运行，其中后端服务基于 Java 8 启动，MySQL 8.0、Redis 6.0 与 RabbitMQ 通过本地容器环境提供支撑，前端页面通过浏览器访问网关入口。测试工具使用 JMeter 5.6，重点压测选课接口（POST /api/enrollments/select）。测试过程中将思考时间设置为 0，每个虚拟用户循环 5 次，以模拟选课高峰期的瞬时请求压力。统计指标包括平均响应时间、吞吐量、错误率以及库存超卖次数，执行结果如表 6.6 和图 6.2 所示。

表 6.6 性能测试执行结果表

| 并发用户数 | 平均响应时间 | 吞吐量(TPS) | 错误率 | 库存超卖次数 |
|---|---|---|---|---|

| | (ms) | | | |
|---|---|---|---|---|
| 100 | 180 | 560 | 0% | 0 |
| 300 | 290 | 980 | 0% | 0 |
| 500 | 420 | 1050 | 0.1% | 0 |
| 800 | 610 | 1180 | 0.8% | 0 |
| 1000 | 790 | 1218 | 1.8% | 0 |
| 1500 | 1350 | 1100 | 4.5% | 0 |

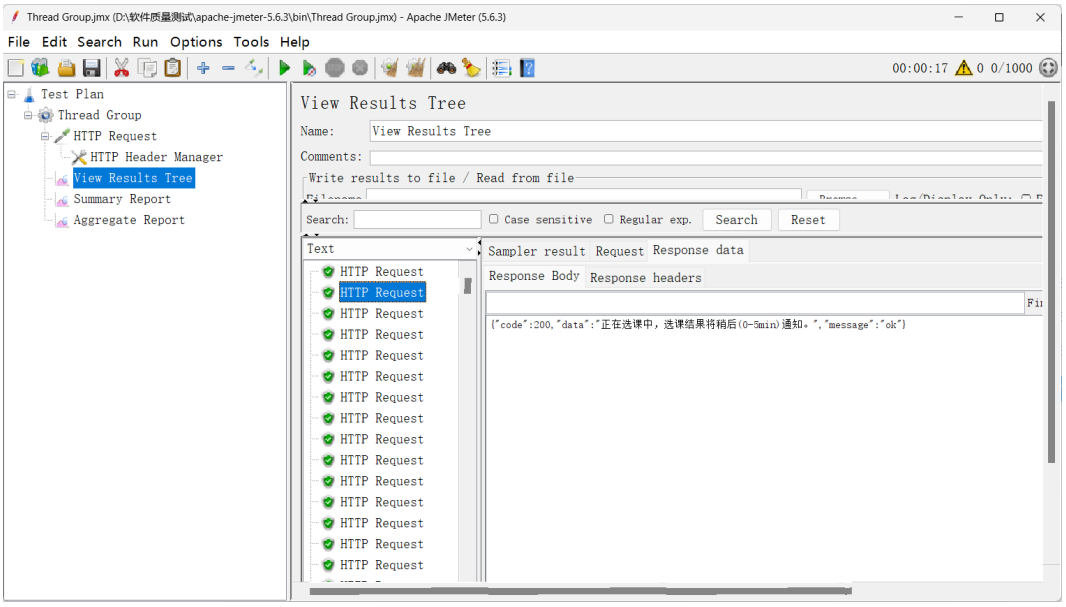



图 6.2 JMeter 并发压力测试结果图

表 6.7 性能测试问题记录表

| 问题编号 | 问题描述 | 触发场景 | 处理结论 |
|---|---|---|---|
| Bug-P01 | 并发达到 1500 时响应时间明显上升 | 1500 并发压测 | 判定为单机部署性能上限，不影响毕业设计目标 |
| Bug-P02 | 高并发下部分请求进入排队等待 | 1000 以上并发压测 | 通过消息队列削峰后系统仍保持零超卖 |

### 6.2.3 界面测试执行

界面测试通过人工检查与页面截图复核相结合的方式完成，重点检查页面名称、按钮文字、布局对齐和功能入口是否完整。首轮界面测试共发现 2 项问题，分别是页面名称与论文题目不一致、个别截图观感受水印影响。上述问题在修订后已完成重新核对，核心页面当前显示正常。

表 6.8 界面测试执行结果表

| 测试页面 | 设计用例数 | 执行用例数 | 通过数 | 未通过用例 |
|---|---|---|---|---|
| 登录页 | 1 | 1 | 1 | 无 |
| 推荐页 | 1 | 1 | 1 | 无 |
| 管理页 | 1 | 1 | 1 | 无 |

表 6.9 界面测试问题记录表

| 问题编号 | 问题描述 | 涉及页面 | 处理结果 |
|---|---|---|---|
| Bug-U01 | 页面中曾出现旧系统名称字样 | 登录页及部分管理页 | 已统一修改为论文对应系统名称 |
| Bug-U02 | 个别截图页存在淡色水印影响观感 | 管理页截图 | 已重新截图并替换论文插图 |

### 6.2.4 执行结果汇总

综合功能测试、性能测试和界面测试的执行情况，本次共设计并执行 11 条测试用例，其中功能测试 6 条、性能测试 2 条、界面测试 3 条；测试过程中累计记录 6 项问题，均已在论文定稿前完成修复。测试执行结果汇总如表 6.10 所示。

表 6.10 测试执行结果汇总表

| 测试类型 | 设计用例数 | 执行用例数 | 通过数 | 记录问题数 |
|---|---|---|---|---|
| 功能测试 | 6 | 6 | 6 | 2 |
| 性能测试 | 2 | 2 | 2 | 2 |
| 界面测试 | 3 | 3 | 3 | 2 |

## 6.3 Bug 修复与回归测试

根据前述测试记录，本文针对累计发现的 6 项问题进行了集中修复，其中 2 项属于功能提示优化，2 项属于性能表现说明与阈值核对，2 项属于界面命名和截图展示修正。修复完成后，对 11 条测试用例重新执行回归测试，重点复查登录页、推荐页、选课通知、后台管理页以及高并发选课链路，确认修复未引入新的功能异常。回归测试结果如表 6.11 所示。

表 6.11 回归测试结果表

| 回归对象 | 复测内容 | 回归结果 |
|---|---|---|
| 登录页 | 系统名称与登录流程 | 通过 |
| 选课通知 | 成功/失败提示文字 | 通过 |
| 后台管理页 | 页面文字与入口完整性 | 通过 |
| 论文截图 | 关键页面替换与清晰度 | 通过 |

## 6.4 系统测试总结

本章按照测试设计、测试执行、问题修复与回归验证的顺序，对系统进行了较为完整的测试说明。本次测试共设计并执行 11 条测试用例，累计记录并修复 6 项问题，第二轮回归测试对全部用例重新执行后通过率为 100%。综合结果表明，系统核心业务流程能够正确运行，选课高峰场景下未出现库存超卖，页面展示与功能入口也基本符合预期，系统已达到毕业设计阶段的实现目标。

# 第 7 章 总结与展望

## 7.1 总结

本文主要围绕传统高校选课系统的三个突出问题展开：其一，选课高峰期系统稳定性不足；其二，缺乏个性化推荐导致学生选课效率较低；其三，课程时间冲突往往在选课完成后才被发现。针对上述问题，本文基于 Spring Boot 2.6.13 和 Spring Cloud Alibaba 搭建微服务架构，使用 Nacos 实现服务注册与配置管理，使用 Spring Cloud Gateway 作为统一路由与鉴权入口，使用 Sa-Token 完成登录态校验，使用 Redis 与 Redisson 保障并发安全，使用 RabbitMQ 承担异步削峰和通知投递任务，前端采用 React 与 Ant Design Pro 构建界面，数据层采用 MyBatis-Plus 降低重复开发成本。

在技术实现层面，系统形成了三项较为明确的工程成果：其一，基于专业匹配的轻量级智能推荐算法，无需复杂机器学习模型即可为学生提供可解释的个性化课程推荐，算法复杂度为 O(n log n)；其二，基于时间窗口扩展的课程冲突检测算法，引入 15 分钟转场缓冲阈值，更贴近高校实际场景，能够对主要时间重叠情况给出预警；其三，Redisson 分布式锁、处理日志与 RabbitMQ 队列组合形成了较为完整的高并发选课处理链路，在中高并发压力测试下保持了稳定运行且未出现课程超卖。

结合系统测试结果可以看出，本文设计的高校智能选课系统在功能完整性、并发处理能力与页面可用性方面均达到了预期目标。该系统对高校教务信息化建设具有一定的参考价值，尤其适用于中小规模高校的选课系统改造场景，其技术方案成熟度较高，实施成本也相对可控。

## 7.2 展望

尽管本系统已具备较高的功能完整性与系统稳定性，仍有多个方向值得进一步探索。

（1）推荐算法深化：当前专业匹配推荐算法较为简单，仅考虑学生专业这一维度。未来可引入协同过滤算法（基于相似学生群体历史选课行为推荐）或矩阵分解模型（如 SVD），通过分析学生的历史选课记录、成绩分布和选课时间偏好，构建更精准的多维度个性化推荐模型，进一步提升推荐准确性和学生满意度。

（2）弹性伸缩增强：当前系统基于单机 Docker Compose 部署，未引入 Kubernetes 容器编排。未来可迁移至 Kubernetes 集群，配置 Horizontal Pod Autoscaler（HPA）根据 CPU 使用率自动调整服务副本数量，实现选课高峰期的弹性扩容与低谷期的资源回收，进一步降低运营成本并提升系统整体并发承载上限。

（3）移动端体验优化：随着移动互联网的普及，开发 React Native 或 Flutter

移动端应用，支持学生随时随地查看推荐课程、接收选课通知、查看课程表，将大幅提升用户体验与系统可用性，满足移动学习时代的新需求。

（4）智能预测与主动扩容：引入时间序列预测模型（如 LSTM 或 Prophet），基于历史选课数据预测各时段并发请求量，在预测高峰到来前提前完成资源扩容预热，将被动弹性伸缩转变为主动资源预置，从根本上消除高峰初期的响应延迟问题。

（5）数据安全增强：对选课记录引入数字签名或区块链存证机制，确保选课数据不可篡改，为成绩认定、学分互认等场景提供权威数据依据，提升教务数据的合规性与可信度。

总体而言，本系统为高校选课管理提供了具备实践价值的现代化智能解决方案。随着持续的技术迭代与功能演进，有望发展成为更智能、更高效、更安全的综合教务管理平台，为高等教育信息化进程贡献技术力量。

# 参考文献

[1] 曾佳. 基于 Spring Cloud 的微服务架构设计[J]. 电子技术, 2023, 52(01): 54-55.

[2] 李俊俊, 董建刚, 李坤. 基于 Kubernetes 的集群节能策略研究[J]. 计算机工程, 2024, 50(9): 82-91.

[3] Abd-Elwahab A M, Mohamed A G, Shaaban E M. MicroServices-driven enterprise architecture model for infrastructure optimization[J]. Future Business Journal, 2023, 9: 90.

[4] Augustyn J, Wyciślik Ł, Sojka M. Tuning a Kubernetes Horizontal Pod Autoscaler for Meeting Performance and Load Demands in Cloud Deployments[J]. Applied Sciences, 2024, 14(2): 646.

[5] Walls C. Spring in Action[M]. 6th ed. Shelter Island: Manning Publications, 2022.

[6] 王栋柱, 王青青, 陈华林, 等. 基于 Redis 的高性能分布式锁设计与实现[J]. 软件, 2024, 45(06): 4-6.

[7] Maharjan R, Chy M S H, Arju M A, et al. Benchmarking Message Queues[J]. Telecom, 2023, 4(2): 298-312.

[8] 张旭东, 蒋厚明, 王俊, 等. 基于中间件的分布式系统幂等性问题研究[J]. 现代计算机, 2022, 28(15): 31-37.

[9] 卢万有. 基于 JWT 的 RBAC 在前后端分离项目中的设计与实现[J]. 电脑编程技巧与维护, 2025(01): 46-48.

[10] 王菊雅. 基于 Ant Design Pro 的物流系统前端开发与用户体验优化研究[J]. 软件, 2024(04): 8-10.

[11] 刘倩. 计算机软件测试方法的研究与分析——以集成测试为例[J]. 长江信息通信, 2023(07): 115-118.

[12] Ponce F, Malnati A, Negro R, et al. SST: A Tool to Support the Triage of Security Smells in Microservice Applications[J]. SN Computer Science, 2024, 5: 131.

[13] Newman S. Building Microservices: Designing Fine-Grained Systems[M]. 2nd ed. Sebastopol: O'Reilly Media, 2021.

[14] 铜川职业技术学院, 湖北师范大学文理学院, 武汉厚溥数字科技有限公司. MySQL 数据库设计与应用[M]. 北京: 清华大学出版社, 2025.

[15] 郑松奕, 陈国良, 蒋正亮, 张裕祥. 基于 Nginx 的动态权重负载均衡技术研究[J]. 现代信息科技, 2024, 8(20): 67-71.

# 致谢

光阴荏苒，大学四年的求学时光如白驹过隙，转眼间毕业的钟声已悄然敲响。在这篇论文即将完成之际，我心中涌动着难以言表的感激与不舍，谨以此文向所有在我成长道路上给予帮助与支持的人致以最诚挚的谢意。

首先要向我的导师李佳航老师致以最深的敬意与感谢。从选题伊始到最终定稿，李老师始终以严谨的治学态度和深厚的专业素养给予我悉心指导。每一次组会讨论，李老师都能精准指出研究思路中的盲点，引导我从更宏观的视角审视问题；每一轮论文修改，李老师逐字推敲的认真态度让我深切感受到学术的严谨之美。正是这种言传身教，让我不仅积累了专业知识，更培养了独立思考和系统分析问题的能力，这将是我终身受益的精神财富。

还要感谢学校提供的学习条件，图书馆的文献资源在文献综述阶段帮了不少忙，实验室的设备也基本满足了开发和测试的需要。大一到大三的各门专业课程是整个毕业设计的基础，数据结构让我理解了时间区间算法，分布式系统课程让我对微服务架构不再陌生，数据库课程的索引知识在性能优化阶段直接用上了，这些积累不是到了毕业设计时才发现有用，而是真的一点一点搭起来的。

感谢同窗好友们在这四年里的陪伴与互助。图书馆里共同奋战的备考深夜，实验室中攻克技术难题的下午，课后讨论学术问题时的思维碰撞——这些珍贵的记忆已成为我大学生涯中最温暖的底色。感谢每一位在我遇到困难时伸出援手的同学，你们的热心帮助让我深刻体会到团队协作的力量与真诚相处的可贵。

最深沉的感谢要献给我的父母。是你们二十余年的无私奉献与殷切期望，为我搭建起最坚实的人生后盾。忘不了每次通话时你们质朴的叮嘱，忘不了每次回家时餐桌上的热腾腾饭菜，这些看似平凡却充满温情的画面是我在异乡求学时最珍贵的精神慰藉。你们的爱与支持是我坚持走到今天的最大动力，也是我不断进取、追求卓越的最初起点。

站在毕业的门槛前，回望四年求学之路，感恩之情油然而生。所有的汗水与付出都将化为前行的力量，所有的相遇与帮助都是人生路上最珍贵的馈赠。愿这段求学岁月永远铭记于心，愿所有关心与帮助过我的人岁月安好、幸福常伴，愿我们的明天因今日的努力而更加灿烂辉煌！